Individual Contribution Report

Student 1 - [S.H.Chehan Mendiya / TG_IT_2025_2500]

A. Assigned Component

I was responsible for the following components of the AuraFlex Gym Management System:

1. Authentication and User Login

2. Member Management

3. Membership Management

These components handle user access, member records, membership assignments, and membership payments.

B. Objective of the Component

The objectives were to:

• Secure the system through user authentication.

• Manage gym member records.

• Assign memberships to valid members.

• Track membership dates, fees, and statuses.

• Connect membership assignments with the payment component.

• Provide role-based access for administrators, receptionists, and trainers.

C. Functions Implemented

-Authentication-

Implemented in auth.c:

• login ()

• Password masking and input handling.

• User credential verification.

• Active-user validation.

• Login attempt control.

-Member Management-

Implemented in member.c:

• memberMenu()• registerMember()

• viewMembers()

• searchMember()

• updateMember()

• freezeMember()

• unfreezeMember()

-Membership Management-

Implemented in membership.c:

• membershipMenu()

• assignMembership()

• viewMemberships()

• recordPayment()

• viewPayments()

• checkMember()

• checkMembershipID()

• readMembership()

• writeMembership()

D. Key Features and Technical Work

- Authentication -

The authentication system:

• Reads user details from users.txt.

• Verifies usernames and passwords.

• Checks whether the user account is active.

• Masks password input using asterisks.

• Allows a maximum of three login attempts.

• Stores the authenticated user’s ID, username, role, and status.

• Provides role-based access to the main dashboard.

- Member Management -

The member component allows the system to:

• Register new members.

• Prevent duplicate member IDs.

• View all members.

• Search for members by ID.

• Update member information.

• Freeze member accounts.

• Unfreeze member accounts.• Store each member's ID, name, phone number, email, and status.

New members are automatically assigned the ACTIVE status.

-Membership Management-

The membership component allows users to:

• Assign a membership to an existing member.

• Prevent duplicate membership IDs.

• Validate that the selected member exists.

• Record membership plan details.

• Store start and end dates.

• Record membership fees.

• Assign an ACTIVE status to new memberships.

• Display membership records.

• Connect membership assignment with payment processing.

When a membership is assigned, the system calls processMembershipPayment() to record the required membership payment.

E. Files / Data Used

The main implementation files are:

• auth.h

• auth.c

• member.h

• member.c

• membership.h

• membership.c

• users.txt

• members.txt

• memberships.txt

• payments.txt

• temp.txt

The components are connected to the main program through main.c.

F. Validation, Searching, Sorting or Calculations Implemented

-Authentication Validation-

• Username and password must match a record in users.txt.

• Only users with ACTIVE status can log in.

• Login attempts are limited to three.

• Invalid credentials generate an error message.

-Member Validation-

• Member IDs must be unique.

• Member records are checked before updates.

• Missing member files are handled with error messages.

• Member status is managed using ACTIVE and FROZEN.

-Membership Validation-

• Membership IDs must be unique.

• A membership can only be assigned to an existing member.

• Membership fees must be greater than zero.

• Membership records are read using a fixed seven-field format.

• Membership status is initially set to ACTIVE.

-Searching-

• Members can be searched using their member ID.

• Existing members are checked before assigning memberships.

• Existing membership IDs are checked before creating new records.

-Sorting-

No sorting algorithm is implemented. Records are displayed in the order stored in the relevant text files.

-Calculations-

The membership component stores and processes:

• Membership fees.

• Start dates.

• End dates.

Payment processing calculates and records the payment amount associated with a membership.

G. Integration with Other Components

• main.c connects authentication with the role-based dashboard.

• The authentication component determines whether the user is an administrator, receptionist, or trainer.

• Member management is used by the membership component to verify member IDs.

• Membership management calls processMembershipPayment() from the payment component.

• The reports component reads members, memberships, and payment records to generate reports.

• The payment component uses member status to ensure payments are recorded only for valid active members.

• Shared interface functions are reused from the system and UI modules.

H. Challenges and Solutions

1. Role-based access control

Challenge:

Different users require different system permissions.

Solution:

The login function stores the authenticated user's role, and main.c displays the appropriate dashboard options for administrators, receptionists, and trainers.

2. Secure password input

Challenge:

Displaying passwords directly would expose user credentials.

Solution:

The authentication component masks password characters with asterisks and supports backspace handling.

3. Limiting failed login attempts

Challenge:

Unlimited login attempts could weaken system security.

Solution:

The system allows only three attempts before terminating the login process.

4. Preventing duplicate member and membership records

Challenge:

Duplicate IDs could create inconsistent records.

Solution:

The system searches the relevant files before adding a new member or membership.

5. Maintaining relationships between members and memberships

Challenge:

A membership should not be assigned to a non-existent member.

Solution:

The checkMember() function verifies the member ID before a membership is created.

6. Updating text-based records

Challenge:

Updating a record in the middle of a text file is difficult.

Solution:

The original data is copied into a temporary file while the updated record is written. The temporary file then replaces the original file.*7. Connecting membership and payment processing*

Challenge:

A membership assignment also requires a payment record.

Solution:

The membership component calls the shared processMembershipPayment() function. If payment processing fails, the membership assignment is cancelled.

Student 2 - [K.R Karanyaka / TG_IT_2025_2498]

A. Assigned Component

- Trainer Management Component

I was responsible for designing and implementing the trainer related functions of the AuraFlex Gym Management System. This component manages trainer records and their relationships with gym members.

B. Objective of the Component

The objective of the trainer component is to allow administrators and trainers to:

- Register new trainers.

- View all registered trainers.

- Search for a trainer using the trainer ID.

- Update trainer information.

- Activate or deactivate trainers.

- Assign members to trainers.

- Remove members from trainers.

- View the members assigned to a specific trainer.

C. Functions Implemented

The main functions implemented in `trainer.c` and declared in `trainer.h` are:

- trainerMenu()
- registerTrainer()
- viewTrainers()
- searchTrainer()
- updateTrainer()
- changeTrainerStatus()
- assignMemberToTrainer()
- removeMemberFromTrainer()
- viewTrainerMembers()
- getMemberTrainer()

D. Key Features and Technical Work

The trainer menu includes these options:

1. Register Trainer
2. View All Trainers
3. Search Trainer
4. Update Trainer Details
5. Activate or Deactivate Trainer
6. Assign Member to Trainer
7. Remove Member from Trainer
8. View a Trainer's Members
9. Return to the Main Menu

Trainer records contain:

- Trainer ID
- Full name

- Phone number

- Specialization

- Current status

New trainers are automatically assigned the `ACTIVE` status during registration.

The component uses temporary files when updating or deleting records. This prevents the original data from being rewritten directly and allows the file contents to be reconstructed safely.

E. Files / Data Used

The main files used by the trainer component are:

- trainer.h -- Contains trainer function declarations.

- trainer.c -- Contains the trainer management implementation.

- trainers.txt -- Stores trainer details.

- trainer_members.txt -- Stores trainer–member assignments.

- `main.c` -- Integrates the trainer menu with the administrator and trainer dashboards.

Temporary files are also used when updating trainer records or removing assignments:

- `data/trainers_temp.txt`

- `data/trainer_members_temp.txt`

F. Validation, Searching, Sorting or Calculations Implemented

Validation

The component validates:

• That trainer IDs, names, phone numbers, specializations and member IDs are single words, because the records are separated by spaces.

• That a trainer ID is not already used when a trainer is registered.

• That a trainer exists before searching, updating, changing the status or assigning a member.

• That members are assigned only to trainers whose status is ACTIVE.

• That a member exists in members.txt before the assignment is saved.

• That the same trainer–member assignment is not saved twice.

• That the assignment to remove actually exists.

• That the menu choice is a valid number and that the status choice is 1 or 2.

• That files can be opened, and that the data files are created if they are missing.

Names and specializations containing spaces must use underscores, for example:

Personal_Training

Searching

Trainers can be searched using their unique trainer ID. The search reads trainers.txt and displays the matching record.

The component also searches for:

• Existing trainer IDs.

• Existing member IDs in members.txt.

• Existing trainer–member assignments.

Sorting

No sorting algorithm is implemented. Trainers are displayed in the same order in which they appear in trainers.txt.

Calculations

The component counts the registered trainers and the members assigned to a trainer, and displays both totals.

G. Integration with Other Components

The trainer component is integrated with the rest of the system through `main.c`.

- Administrators can access the complete trainer management menu.
- Trainers can view their assigned members.

- Trainers can assign members and remove members from their assignments.

- The member data file is checked before a member is assigned.

- The trainer status affects assignment functionality.

- The common UI functions, such as screen clearing and logo display, are reused from `ui.h`.

- Trainer records and member assignments are stored persistently in text files.

H. Challenges and Solutions

1. Storing records in text files

Challenge:

The system uses text files instead of a database, so records must be manually read, updated, and rewritten.

Solution:

I used temporary files such as trainers_temp.txt and trainer_members_temp.txt. The updated records are written to the temporary file, after which the original file is replaced.

2. Preventing duplicate records

Challenge:

Registering the same trainer more than once could create duplicate records.

Solution:

The trainerIdExists() function checks the trainer file before registration and prevents duplicate trainer IDs.

3. Maintaining valid trainer–member relationships

Challenge:

A member could be assigned to a trainer who does not exist or is inactive.

Solution:

The system verifies both the trainer and member IDs. It also checks that the trainer status is ACTIVE before allowing an assignment.

4. Handling whitespace-separated data

Challenge:

The text-file format uses spaces to separate fields, so names or specializations containing spaces could break the record format.

Solution:

Input validation prevents spaces in individual fields and instructs users to use underscores instead.

5. Removing assignments safely

Challenge:

Deleting a single assignment from a text file requires rewriting the remaining records.

Solution:

The matching assignment is skipped while all other assignments are copied to a temporary file. The temporary file then replaces the original assignment file.

6. Connecting different user roles

Challenge:

Administrators and trainers require different access levels.

Solution:

The trainer functionality is connected to the role-based dashboard in main.c. Administrators receive full trainer management access, while trainers receive access to member assignment functions.

Student 3 - [Pasindu Madusanka / TG_IT_2025_2485]

A. Assigned Component

Payment Management Component

I was responsible for implementing the payment management functionality of the AuraFlex

Gym Management System. This component records member payments, validates payment

information, searches payment history, and calculates payment totals.

B. Objective of the Component

The objective of the payment component is to allow the system to:

• Record payments made by active members.

• Process membership payments.

• Generate unique payment IDs.

• Validate payment dates and amounts.

• Support different payment methods.

• Search payment history by member ID.

• Display all payment records.

• Calculate the total number and value of payments.

C. Functions Implemented

The main functions implemented in payment.c and declared in payment.h are:

• paymentRecord()

• processMembershipPayment()

• searchPaymentHistory()

• viewPaymentHistory()

• calculateTotalPayments()

• paymentMenu()

Supporting functions were implemented for:

• Reading text input.

• Clearing the input buffer.

• Validating dates.

• Checking whether a member exists.

• Reading payment records.

• Checking duplicate payment IDs.

• Generating payment IDs.

• Saving payment information to the data file.

D. Key Features and Technical Work

The payment menu provides the following options:

1. Record Payment

2. Search Payment History

3. View All Payments

4. Calculate Total Payments

5. Return to Main Menu

Each payment record contains:

• Payment ID.

• Member ID.

• Payment amount.

• Payment date.

• Payment method.

Payment Recording

The system verifies the member ID and confirms that the member is active before recording the payment. It also checks that the amount is greater than zero.

Membership Payment Processing

The processMembershipPayment() function allows the membership component to record a payment when a member purchases or renews a membership.

Payment ID Generation

Payment IDs are generated automatically in the format:

P001

P002

P003

The system checks existing records and generates the next available ID.

Payment Methods

The accepted payment methods are:

• CASH

• CARD

• ONLINE

The input is converted to uppercase before it is stored.

E. Files / Data Used

The main implementation files are:

• **payment.h**

Contains payment function declarations.

• **payment.c**

Contains the payment management implementation.

• **payments.txt**

Stores payment records.

• **members.txt**

Used to verify the existence and active status of members.

• **main.c**

Connects the payment menu to the administrator and receptionist dashboards.

F. Validation, Searching, Sorting or Calculations Implemented

Validation

The component validates:

• Empty member IDs.

• Member ID existence.

• Member account status.

• Payment amounts greater than zero.

• Payment date format.

• Valid calendar dates.

• Leap-year dates.

• Accepted payment methods.

• Numeric menu selections.

• Duplicate payment IDs.

• File opening and writing errors.

Payment dates must use the following format:

YYYY-MM-DD

For example:

2026-10-31

Searching

Payment history can be searched using a member ID. All payments belonging to that member are displayed.

Sorting

No sorting algorithm is implemented. Payment records are displayed in the order stored in payments.txt.

Calculations

The component calculates:

• Total number of payment records.

• Total amount of all recorded payments.

Each valid payment amount is added to the total payment value.

G. Integration with Other Components

The payment component is integrated with the rest of the system through main.c.

• Administrators can access the complete Payment Management menu.

• Receptionists can record payments directly from their dashboard.

• The membership component can call processMembershipPayment() to save membership payments.

• The member data file is checked before a payment is accepted.

• The reports component reads payment data to generate payment reports and summary reports.

• Payment records are stored persistently in payments.txt.

H. Challenges and Solutions

1. Preventing payments for invalid members

Challenge:

A payment should not be recorded for a member who does not exist or is inactive.

Solution:

The paymentMemberExists() function searches members.txt and verifies that the member has an active status.

2. Generating unique payment IDs

Challenge:

Manually entering payment IDs could result in duplicates.

Solution:

The system scans existing payment records, finds the highest payment number, and generates the next available ID automatically.

3. Validating payment dates

Challenge:

Users may enter invalid dates such as incorrect months, days, or leap-year dates.

Solution:

The validDate() function checks the date format, year, month, day range, month length, and leap-year rules.

4. Handling different payment methods

Challenge:

Users may enter payment methods using different letter cases.

Solution:

The input is converted to uppercase and checked against the accepted methods: CASH, CARD,

and ONLINE.

5. Sharing payment functionality with memberships

Challenge:

Membership payments and normal payments must use the same storage and validation logic.

Solution:

The shared savePayment() function is used by both paymentRecord() and processMembershipPayment().

6. Calculating payment totals

Challenge:

Payment values must be read from text records and added accurately.

Solution:

The system reads each valid payment record, increments the payment count, and adds the amount to the running total.

Student 4 - [R.Indumin Sasindu / TG_IT_2025_2493]

A. Assigned Component

Attendance Management Component

I was responsible for implementing the attendance management functionality of the AuraFlex Gym Management System. This component records member check-ins and check-outs and provides attendance history and daily summaries.

B. Objective of the Component

The objective of the attendance component is to:

• Record when active members enter the gym.

• Record when members leave the gym.

• Prevent duplicate check-ins on the same day.

• Search attendance records by member ID.

• Display complete attendance history.

• Provide a daily attendance summary.

C. Functions Implemented

The main functions implemented in attendance.c and declared in attendance.h are:

• attendanceMenu()

• recordCheckIn()

• recordCheckOut()

• searchAttendance()

• viewAttendanceHistory()

• dailyAttendanceSummary()

Supporting functions were implemented for:

• Obtaining the current date.

• Obtaining the current time.

• Checking whether a member exists and is active.

• Reading attendance records.

• Generating attendance IDs.

• Finding an open check-in for the current day.

• Pausing the screen between operations.

D. Key Features and Technical Work

The attendance menu provides the following options:

1. Record Check-In

2. Record Check-Out

3. Search Attendance by Member ID

4. View Attendance History

5. Daily Attendance Summary

6. Return to Dashboard

Each attendance record contains:

• Attendance ID.

• Member ID.

• Attendance date.

• Check-in time.

• Check-out time.

Check-In

When a member checks in, the system:

• Verifies that the member exists.

• Confirms that the member has an ACTIVE status.

• Checks whether the member is already checked in that day.

• Generates a unique attendance ID.

• Automatically records the current date and time.

• Stores - as the check-out value until the member leaves.

Check-Out

When a member checks out, the system:

• Verifies that the member is active.

• Searches for an open attendance record from the current day.

• Records the current time as the check-out time.

• Rewrites the attendance file using a temporary file.

Daily Summary

The daily summary displays:

• Total check-ins.

• Total members who checked out.

• Members currently inside the gym.

E. Files / Data Used

The main implementation files are:

• attendance.h

Contains attendance function declarations.

• attendance.c

Contains the attendance management implementation.

• attendance.txt

Stores attendance records.

• members.txt

Used to verify member IDs and active member status.

• main.c

Connects attendance management to the administrator and receptionist dashboards.

A temporary file named attendance_temp.txt is used when updating check-out information.

F. Validation, Searching, Sorting or Calculations Implemented

Validation

The component validates:

• Whether the member ID exists.

• Whether the member is active.

• Whether the member is already checked in today.

• Whether a check-out record exists for the current day.

• Whether the attendance file can be opened.

• Whether menu input is numeric and within the valid range.

Frozen or non-existent members cannot check in or check out.

Searching

Attendance records can be searched using a member ID. The system displays all matching attendance records, including:

• Attendance ID.

• Date.

• Check-in time.

• Check-out time.

The system also searches for an open attendance record when processing a check-out.

Sorting

No sorting algorithm is implemented. Attendance records are displayed in the order stored in attendance.txt.

Calculations

The daily summary calculates:

• Total check-ins for the current date.

• Total completed check-outs.

• Number of members currently checked in.

The current date and time are generated using the C time.h library.

G. Integration with Other Components

• main.c connects attendance management to the main dashboard.

• Administrators can access the complete attendance management menu.

• Receptionists can record attendance and access attendance functions.

• The attendance component uses members.txt to validate member identity and status.

• Shared interface functions are reused from the system and UI modules.

• Attendance data is stored persistently in attendance.txt.

H. Challenges and Solutions

1. Preventing check-ins for invalid members

Challenge:

Attendance should only be recorded for registered and active members.

Solution:

The memberExists() function searches members.txt and verifies that the member status is ACTIVE.

2. Preventing duplicate daily check-ins

Challenge:

A member should not have multiple open check-in records on the same day.

Solution:

The findTodayAttendance() function searches for an attendance record with the current date and a - check-out value before allowing a new check-in.

3. Updating check-out records in a text file

Challenge:

Text files do not support directly editing a record in the middle of the file.

Solution:

All records are copied to attendance_temp.txt, the matching record is updated, and the temporary file replaces the original file.

4. Generating unique attendance IDs

Challenge:

Manually entering attendance IDs could lead to duplicates.

Solution:

The system scans existing attendance records, identifies the highest numeric ID, and generates the next ID in the format A001, A002, and so on.

5. Recording accurate attendance times

Challenge:

Users should not manually enter attendance dates and times.

Solution:

The system automatically obtains the current date and time using the C time library.

6. Identifying members currently inside the gym

Challenge:

The system must distinguish completed visits from open visits.

Solution:

An attendance record with - as its check-out value is treated as an active visit, while a recorded check-out time indicates that the member has left.

Student 5 - [T.V.K.Sendiya / TG_IT_2024_2120]

A. Assigned Component

Reports and Reporting Component

I was responsible for implementing the reporting section of the AuraFlex Gym Management System. This component collects information from member, membership, and payment files and presents it in a readable report format.

B. Objective of the Component

The objective is to provide administrators with an overview of the gym's operations, including:

• Registered members.

• Membership records.

• Expired memberships.

• Payment transactions.

• Overall system statistics.

C. Functions Implemented

The main functions implemented in reports.c and declared in reports.h are:

• reportsMenu()

• memberReport()

• membershipReport()

• expiredMembershipReport()

• paymentReport()

• summaryReport()

Supporting functions were also implemented for:

• Obtaining the current date.

• Comparing membership expiry dates.

• Clearing invalid input.

• Pausing the screen between reports.

The member and membership report options reuse the existing viewMembers() and viewMemberships() functions.

D. Key Features and Technical Work

The reports menu provides the following options:

1. Member Report

2. Membership Report

3. Expired Membership Report

4. Payment Report

5. Summary Report

6. Return to Main Menu

Member Report

Displays the available member records, including member IDs, names, contact details, and account status.

Membership Report

Displays membership records and their current membership information.

Expired Membership Report

Checks active memberships and compares their end dates with the current system date. Memberships with an end date earlier than today are displayed as expired.

Payment Report

Displays:

• Payment ID.

• Member ID.

• Payment amount.

• Payment date.

• Payment method.

• Total payment records.

• Total recorded payment amount.

Summary Report

Provides a combined overview containing:

• Total members.

• Total membership records.

• Active and non-expired memberships.

• Expired active memberships.

• Cancelled memberships.

• Total payment records.

• Total recorded payment amount.

E. Files / Data Used

The main implementation files are:

• reports.h

Contains report function declarations.

• reports.c

Contains the reporting logic and report menu.

The data files used are:

• members.txt

• memberships.txt

• payments.txt

The component also uses existing member and membership functions from:

• member.h

• membership.h

F. Validation, Searching, Sorting or Calculations Implemented

Validation

The reports component validates:

• Numeric menu input.

• Invalid menu selections.

• File availability before reading records.

• Correct record formats while using fscanf().

If a data file cannot be opened, an error message is displayed instead of silently continuing.

Searching and Filtering

The expired membership report filters records by checking:

• Membership status is ACTIVE.

• Membership end date is earlier than the current date.

The summary report filters memberships into:

• Active and non-expired.

• Expired.

• Cancelled.

Sorting

No sorting algorithm is implemented. Records are displayed in the same order in which they appear in the data files.

Calculations

The following calculations are performed:

• Count of total members.

• Count of total memberships.

• Count of active memberships.

• Count of expired memberships.

• Count of cancelled memberships.

• Count of payment records.

• Total payment amount.

The current date is generated using the C time library and formatted as:

YYYY-MM-DD

G. Integration with Other Components

The reports component is integrated with the administrator dashboard in main.c.

• Administrators access reports through the Reports option.

• The reports use member data managed by the member component.

• Membership reports use data created by the membership component.

• Payment reports use payment records created by the payment component.

• Shared interface functions such as screen clearing and logo display are reused from ui.h.

• Report data is read directly from the system's text-based data files.

H. Challenges and Solutions

1. Reading data from multiple files

Challenge:

The summary report requires information from members, memberships, and payments.

Solution:

Each file is opened and processed separately. Counters and totals are maintained while reading the records.

2. Detecting expired memberships

Challenge:

Membership expiry depends on the current date.

Solution:

The system obtains the current date using time.h, formats it as YYYY-MM-DD, and compares it with membership end dates.

3. Handling missing data files

Challenge:

A missing file could cause the report to fail unexpectedly.

Solution:

Each file is checked after opening. If the file cannot be opened, a clear error message is

displayed and the report stops safely.

4. Calculating payment totals

Challenge:

Payment values must be accumulated while processing text records.

Solution:

Each valid payment amount is added to totalPaymentAmount, while the number of valid payment records is tracked separately.

5. Avoiding duplicated functionality

Challenge:

Member and membership viewing functionality already existed in other components.

Solution:

The reporting menu reuses viewMembers() and viewMemberships() instead of duplicating their implementation.